

学習済みの強化学習AIをゲームに埋め込んで、ブラウザで対戦できるようにした

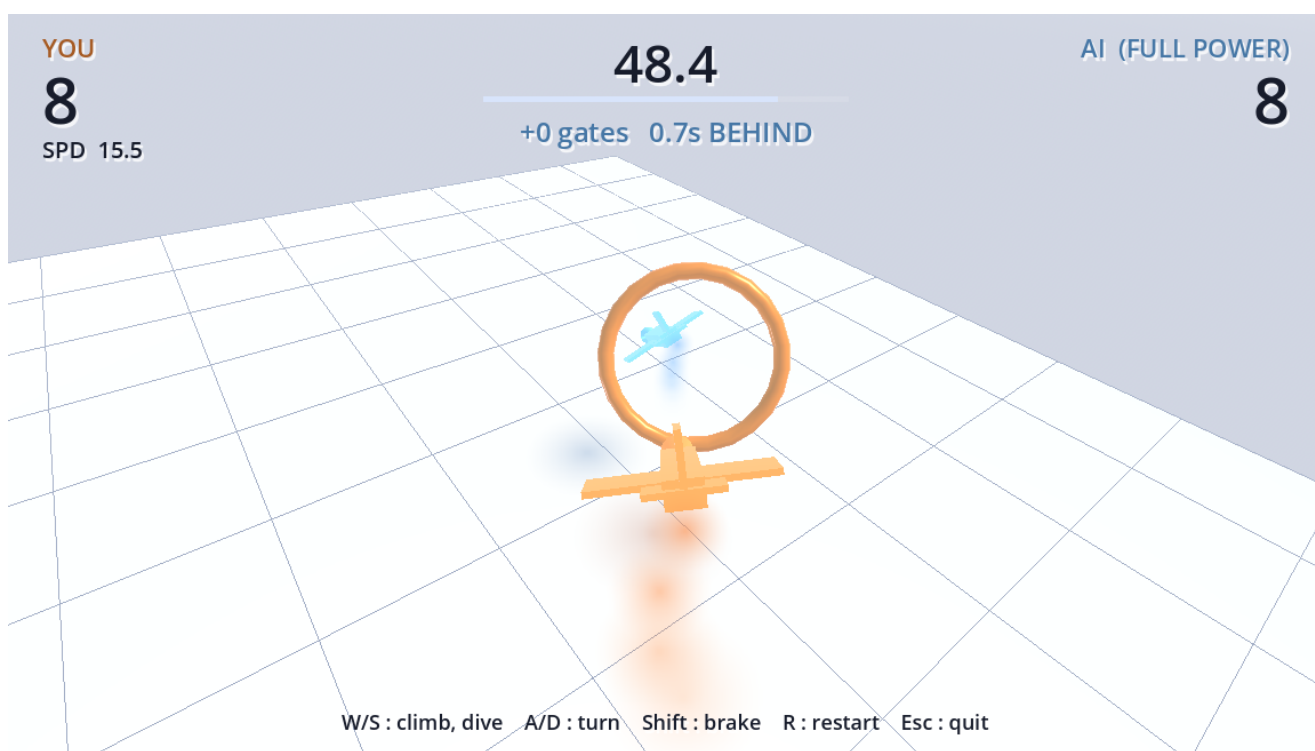
Godot 4.4 + PPO / 2026 年 8 月 19 日

<https://ms022019.github.io/fly-by-3d/touch/>

Godot 4.4 + PPO で 3D ゲームを 2 本作り、**学習済み AI の重みをゲーム本体に埋め込んで**、ブラウザで対戦できるところまで持っていきました。Python もサーバーも動いていません。スマホでも遊べます。

遊べます → <https://ms022019.github.io/fly-by-3d/touch/> (PC 版:

<https://ms022019.github.io/fly-by-3d/> / ソース: <https://github.com/ms022019/fly-by-3d>)



作りながら「そもそも強化学習は何をしているのか」を一つずつ確かめたので、その理解の過程も後半にまとめました。

1 何を作ったか

ゲーム	内容	学習前	手書きの方策	PPO
Fly By	空中のリングを連続でくぐる	0.0	34.4	39.6
Ball Collector	球を転がしてターゲットを集める	0.2	36	64

いずれも「60 秒あたりの獲得数」です。

手書きの方策と必ず比べているのが肝でした。これが無いと「AI が 39.6 個」という数字を見ても、それが上手いのか下手なのか判断できません。実際 Fly By では、単純に次のリングへ機首を向けるだけの十数行のコードが 34.4 個取ります。学習した AI はそれを 5 個上回っているだけ、という見方もできるわけです。

環境は CPU 12 コアのみ、GPU なし。学習は 60 万ステップで **3.4 分**でした。

2 強化学習で何をしているのか

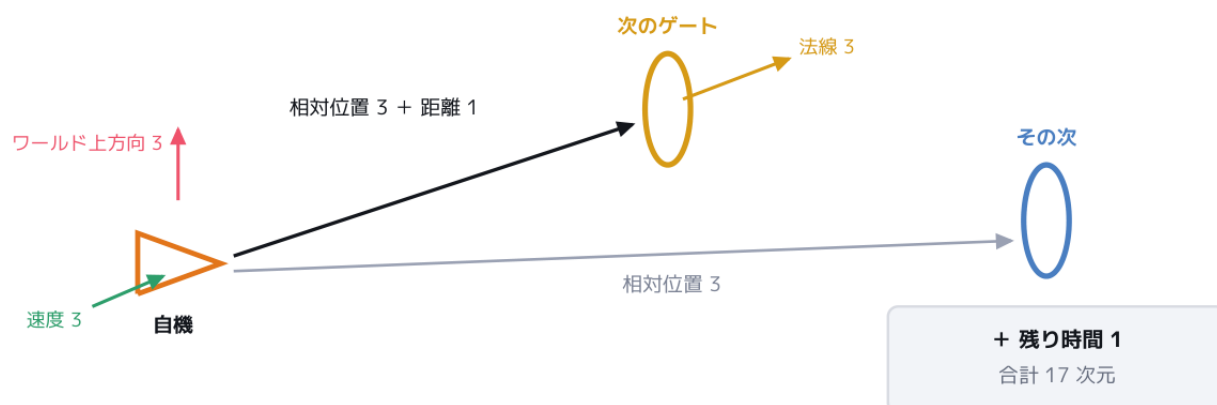
正解のプレイデータは 1 件も与えていません。AI が自分で操縦してみて、点が入った動きの確率を上げ、墜落につながった動きの確率を下げる、という更新をくり返すだけです。



■ AI に見せているもの（観測 17 次元）

画面は見せていません。渡しているのは 17 個の数値だけです。

すべて機体から見た向きに直してから渡す



観測	次元	なぜ必要か
自機の速度	3	曲がりきれぬかは今の速度で決まる

観測	次元	なぜ必要か
次のリングへの相対位置	3	どちらへ向かうかの主信号
次のリングの法線	3	リングには表裏がある
その次のリングへの相対位置	3	手前で減速する判断ができる
ワールド上方向（機体から見た向き）	3	今どれだけ傾いているか
次のリングまでの距離	1	近さを明示的に渡す
残り時間	1	下記

すべて機体から見た向きに変換しています。これをしないと「リングが右 10m」という同じ状況が、機首の向き次第で $(+10,0,0)$ になったり $(-10,0,0)$ になったりして、**同じ判断を向きごとに覚え直す**羽目になります。

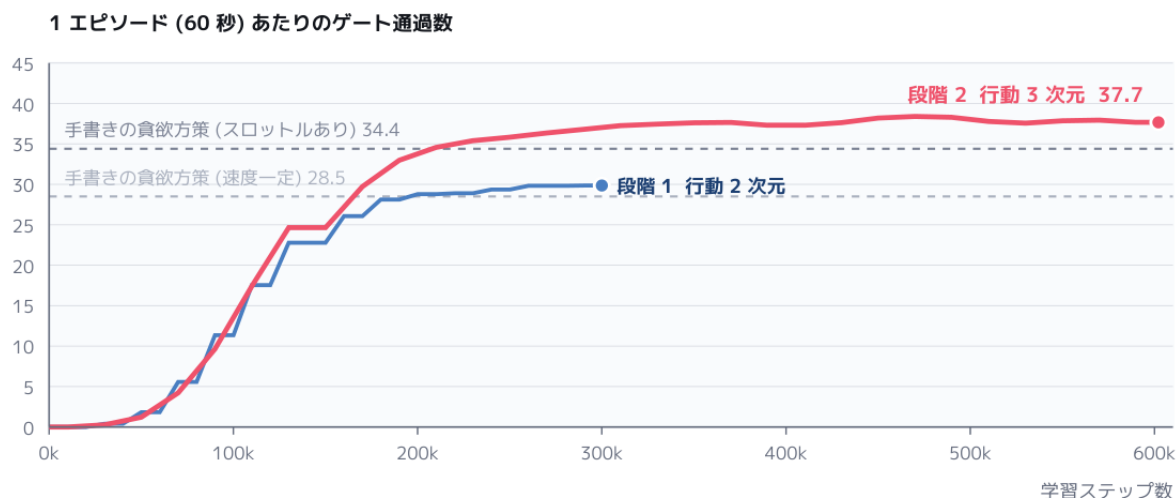
残り時間を入れているのは、エピソードが 60 秒で必ず終わるからです。見せないと AI から見た世界は「まったく同じ状況なのに、あるときは続き、あるときは唐突に終わる」ものになり、「この状態はこの先どれくらい得点につながるか」という見積もりが正しく学習できません。

■ 報酬

事象	報酬
リング通過	+1.0
墜落・コース外	-1.0
くぐらずに面を通り過ぎた	-0.2
次のリングに近づいた	近づいた距離 $\times$ 0.02

最後の距離シェーピングが決定的でした。「くぐったら +1」だけだと、学習開始直後の AI は偶然リングを通り抜けるまで何の手がかりも得られません。でたらめに飛ぶ機体が直径 5m の輪を通る確率は低く、報酬がほぼ全ステップ 0 のまま時間だけが過ぎます。

■ 実際の学習カーブ



いきなり全部の操作を与えず 2 段階に分けました。段階 1 は速度一定でピッチとヨーだけ（行動 2 次元）、段階 2 でスロットルを追加（行動 3 次元）です。

段階	通過数	見て分かる変化
学習前	0.0	でたらめに機首を振り、地面か場外へ突っ込む
5 万	1.2	リングの方を向いて飛ぶ。通るのはまだ偶然
9 万	9.7	狙って通り始める。速度を出しすぎて曲がりきれない
17 万	29.7	手書き方策に並ぶ。墜落がほぼ無くなる
21 万	34.6	カーブ手前で減速するようになる
45 万	38.2	減速量と再加速のタイミングが洗練される

段階 1（速度一定）では AI 31.3 に対し手書き 28.5 とほぼ差がありませんでした。スロットルを与えた段階 2 で 39.6 対 34.4 と差が開きます。AI が上回っているぶんの中身は、ほぼ速度の使い分けでした。「次の次のリング」を観測に入れてあるので、まだ見えていないカーブのきつさを先読みして手前から緩める、という単純な追尾では出てこない振る舞いが現れています。報酬として明示的に教えたものではありません。

3 作りながら確かめたこと

ここからは、実装しながら一つずつ潰していった疑問です。同じところで引っかかる人がいると思うので残します。

■ ロールアウトって何？

方策を更新する前に「とりあえず動かして経験を貯める」その 1 回ぶんです。

1. 今の方策のまま、48 機を 64 ステップ動かす

2. その間の（観測・行動・報酬）**3,072 件**を貯める ← これが 1 ロールアウト
3. 貯めた 3,072 件で方策を更新し、経験は捨てて 1 に戻る

■ バッチは？

貯めた 3,072 件を、**一度に何件ずつ使って重みを更新するか**です。batch_size=256 なので 12 個のミニバッチに切り、それを 10 周 (n_epochs=10) 使います。**1 ロールアウトあたり 120 回**の更新です。

全部を一度に使わないのは、細かく刻んで何度も更新した方が学習が進むから。逆に細かすぎると 1 回ぶんの見積みりが偏ってブレます。

■ 1 ステップごとに報酬が決まって学習される？

報酬は**毎ステップ決まります**が、学習はそこでは起きません。

そしてもう 1 つ大事な点があります。**ある行動の良し悪しは、その場でもらった報酬だけでは判断しません。**そうしないと「今リングをくぐるために減速する」のような、目先は損でも後で得する行動が学習できないからです。減速した瞬間の報酬はむしろ下がりますが、その後 2~3 秒でリングを通れるので、まとめて見れば得になります。

■ アドバンテージの計算、結局 1 ステップぶんでは？

これは自分でも引っかけかりました。TD 誤差の式はたしかに 1 ステップぶんです。

$$\delta = r + \gamma \cdot V(\text{次の状態}) - V(\text{今の状態}) \quad \gamma = 0.99$$

ただし使うのは δ 単体ではなく、**その先の δ を足し合わせたものです** ($\gamma\lambda = 0.99 \times 0.95 = 0.94$)。

$$A_t = \delta_t + 0.94 \cdot \delta_{t+1} + 0.88 \cdot \delta_{t+2} + 0.83 \cdot \delta_{t+3} + \dots$$

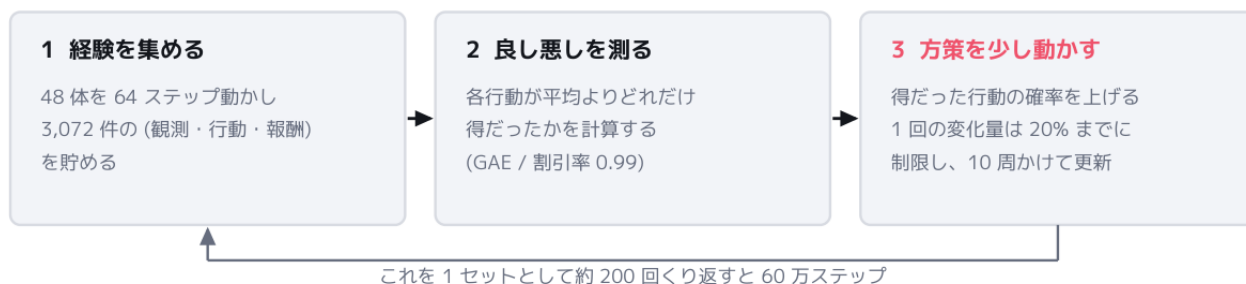
つまり **t 番目の行動の評価には、t+1、t+2、… の報酬が実際に入っています**。1 ステップの式を、64 ステップぶん後ろから前へ積み上げているだけです。だから「64 ステップ貯めてから」でないと計算できません。

では **V(次の状態)** は何のためにあるのか。**窓の外を埋めるため**です。64 番目の行動には未来の実測が 1 つもないので、そこから先は **V** で見積みります。**V を一緒に学習しているのは、この「窓の外」を担当させるため**です。

重み 0.94 の n 乗なので、実質の視野はおよそ **17 ステップ**。1 ステップ = 8 物理フレーム = 0.13 秒なので、**約 2.2 秒先**まで見ています。速度 14 m/s なら約 30m 先 — ちょうど次のリング 1~2 個ぶんです。

「カーブ手前で減速する」が学習できたのは、**この 2.2 秒の窓に「減速した損」と「曲がりきってリングを通った得」の両方が収まっていたから**でした。窓がもっと短ければ、減速はただの損にしか見えません。

■ PPO の「1 回の変化率は 20% まで」って何の 20%？



「その状況でその行動を選ぶ確率」が 1.2 倍を超えて上がらない、という意味です。行動の値そのものが 20% までしか変わらない、ではありません。

1. あるカーブ手前で、方策が「スロットル -0.6 (減速)」を出した
2. 結果は良かった ($A > 0$) → 次から同じ場面でもっと減速しやすくしたい
3. でもその行動を出す確率は、更新前の 1.2 倍まで

正確には「上限で頭打ちになる」というより「それ以上動かす動機が消える」(勾配がゼロになる)という止め方です。比較相手の方策はロールアウト開始時点で固定されるので、この 1.2 倍は 120 回ぶんの更新をまとめた話になります。

たまたま上手くいった経験に飛びついて方策を大きく書き換えると、それまでできていたことまで壊れて成績が崩壊します。一気に変えず、確かめながら少しずつ動かす——それがこの制限の狙いです。

4 学習済み AI をゲームに埋め込む

ここが今回いちばん面白かったところです。



ONNX ランタイムは使わない (プラグインの推論部は C# 実装で、この環境の Godot では動かない)

普通なら ONNX ランタイムを使いますが、godot-rl 同梱の推論部は C# 実装で、この環境の Godot (非 .NET ビルド) では動きません。行き詰まりに見えますが、**そもそもこの方策は 17 → 64 → 64 → 3 の MLP でしかありません。**

```
h = tanh(W0 · obs + b0)  17 → 64
h = tanh(W1 · h + b1)    64 → 64
出力 = W2 · h + b2      64 → 3
```

行列積 3 回と `tanh` だけ。重みは **5,507 個**、base64 にして **31 KB**。それなら重みを GDScript の定数として埋め込み、自分で行列積を回す方が話が早い（実装は 120 行ほど）。

ニューラルネットは基本すべて行列演算に落ちますが、MLP が特別なのは**それ以外に何も無い**ことです。CNN や Transformer だと畳み込みのメモリ配置、Attention、各種正規化層…と扱うものが一気に増えます。**小さな MLP で足りるタスクに設計したことが、そのまま「ゲームに埋め込める」という結果につながりました。**

■ 移植が正しいかをどう確かめたか

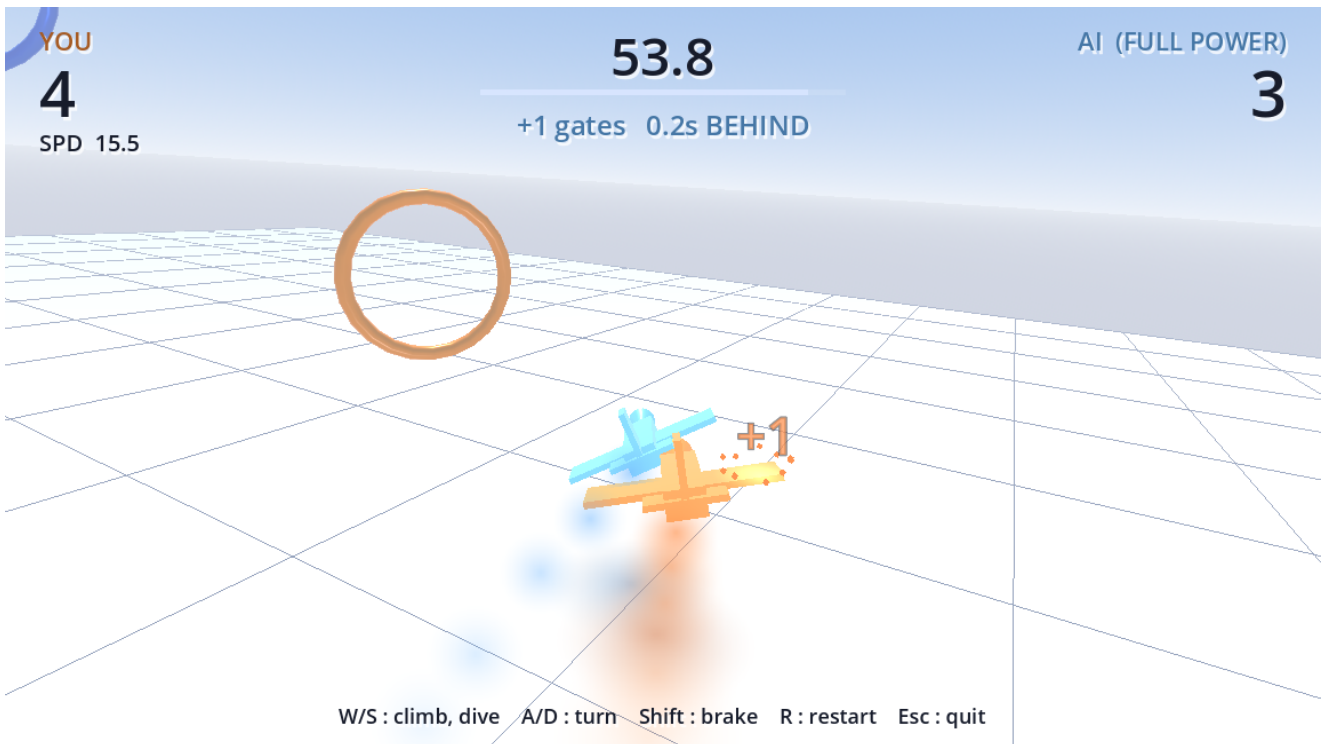
「だいたい動いているように見える」で済ませると、重みの並び順や活性化関数を間違えていても気づけません。そこで **Python と Godot に同じ観測を入れて、出力を突き合わせました。**

```
python tools/export_policy.py --game flyby # 重みを書き出し、参照値を表示
godot --headless --path game --policy-probe # ゲーム内で同じ入力を通した値
```

比較するのは**クリップ前の生の値**にします。`[-1, 1]` に丸めた後だと、飽和して差が隠れてしまうためです。

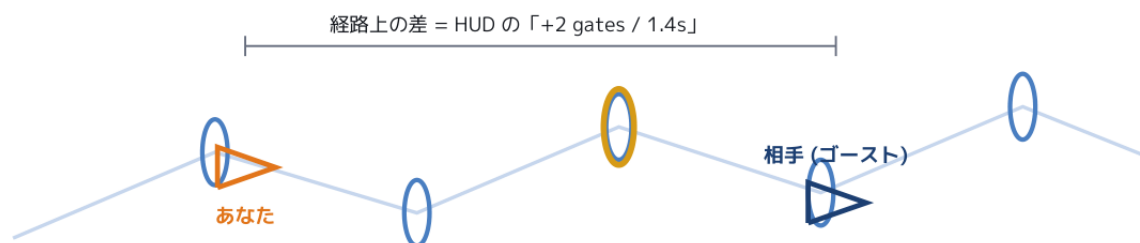
結果は Fly By が完全一致、Ball Collector は float32 の丸め誤差のみ。さらにゲーム内 AI を 8 レース走らせたスコアも **39.25 個**で、Python 推論の 39.6 個と一致しました。

5 対戦モード



■ ゴースト方式にした理由

Ball Collector の対戦は「同じフィールドでターゲットを取り合う」形でした。ところが Fly By はリングを順番にくぐる競技なので、この手が使えません。ゲートを共有すると、AI が先行した瞬間に人間の目標ゲートまで前へ送られてしまい、離されたら 1 個もくぐれなくなります。



実際にはまったく同じ形のコースが 2 本、同じ座標に重なっている

乱数シードを揃えて形を一致させ、当たり判定はレイヤーで分離する。相手のリングと地面は描かない。

そこでまったく同じ形のコースを 2 本、同じ座標に重ねて置きました。乱数シードを揃えるので形は完全に一致し、当たり判定はレイヤーで分けてあるので互いのリングには反応しません。相手のリングと地面は描かないため、画面上はコースが 1 本あるように見え、そこを相手が一緒に飛んでいるように見えます。

この方式の利点は、[course.gd](#) の「目標ゲートの添字」まわりを一切触らずに済むことでした。1 コース 1 機という既存の前提がそのまま保て、シングルプレイと学習パイプラインが壊れません。

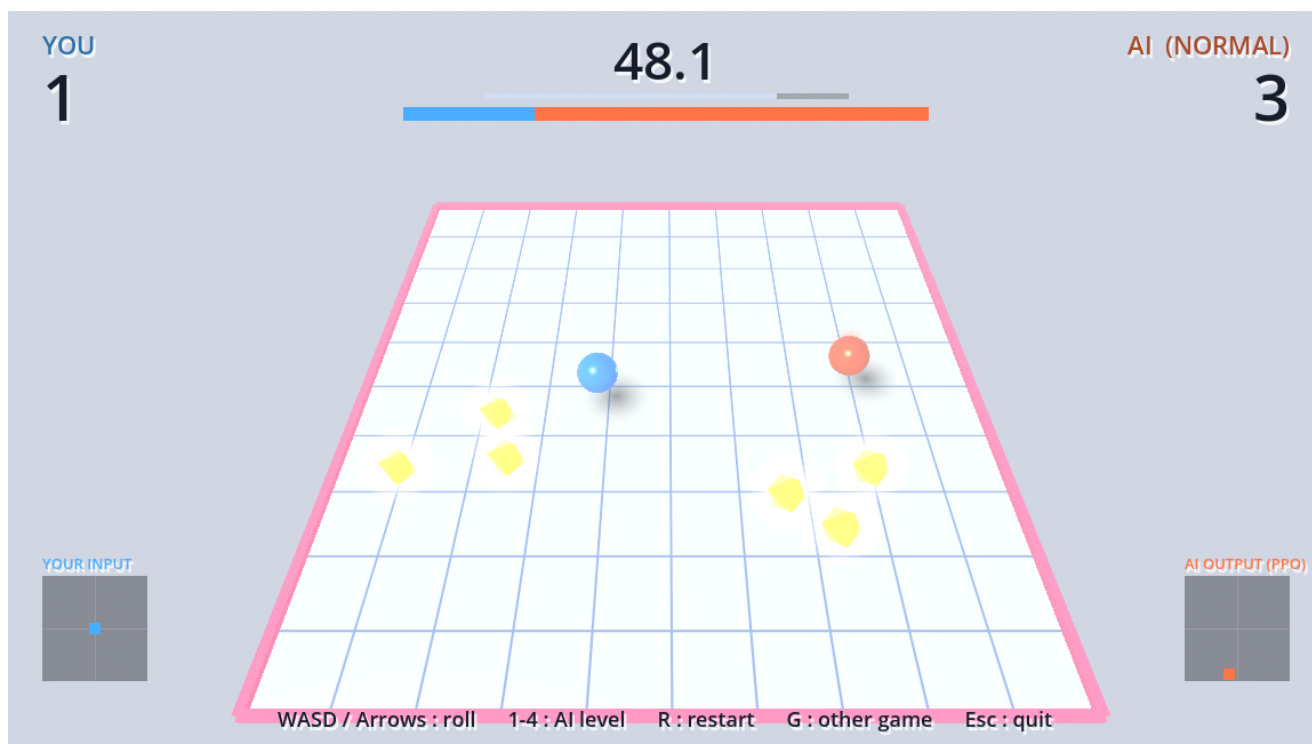
■ AI の強さの絞り方

ニューラルネットには一切触っていません。外側のパラメータだけを絞っています。

レベル	速度上限	行動ノイズ	実測
EASY	6 m/s	0.35	15.4
NORMAL	9 m/s	0.20	22.5
HARD	12 m/s	0.10	28.9
FULL POWER	制限なし	0	39.3

前作のやり方がそのままでは効きませんでした。Ball Collector は「トルクと速度上限を skill 倍」で 4 段階が作れましたが、Fly By で同じことをすると弱くなりません。この機体は旋回の角速度が速度によらず一定なので、遅くすると小回りが利いて、むしろカーブを曲がりきれるようになってしまいます。

さらに罫が 1 つ。観測に含まれる自機の手加減は最高速で割って正規化してあるので、手加減のつもりで最高速そのものを下げると、正規化の基準まで変わり、方策から見える世界が歪みます。観測用と手加減用は別の変数に分ける必要がありました。



6 スマホ対応



キーボード専用だったので、スマホでは読み込めても操作できませんでした（タイトルで SPACE が押せず開始すらできない）。タッチ画面のときだけ画面上の操作 UI を出すようにしています。

- 画面**左半分**をドラッグで操縦、**右半分**でブレーキ（Ball Collector は 2 軸だけなので画面全体がスティック）
- スティックは**触れた場所に出る**方式。固定位置だと端末の大きさや持ち方で指が届かない

- マルチタッチを扱うので Button ではなく生のタッチイベントを見ている（旋回しながらブレーキを踏む同時操作が要するため）

■ 実際に遊んでもらって直した 2 点

タッチだけが不利になっていた。 キーボードは押した瞬間に入力が全開（±1）になるのに、タッチは指を 120px 動かして初めて全開でした。可動半径を 72px に縮め、応答カーブ（0.65 乗）を入れて、20px の移動で 0.17 → 0.43 まで効くようにしました。

時間切れの直後に次のレースが始まってしまう。 操縦していた指を離れた拍子にタップ判定が入っていました。結果表示から 1.2 秒はタップ開始を受け付けないようにしています。

なお `DisplayServer.is_touchscreen_available()` はブラウザによって **false** を返します。これに頼るとスティックが一切作られず操作不能になるので、UI は常に作っておき、**実際にタッチが来た時点で有効化する形**にしました。

7 公開（GitHub Pages）

サーバー側の処理が一切ない静的ファイルだけなので、置ける場所ならどこでも動きます。Web ビルドはスレッドを無効にして書き出してあるため、**COOP / COEP** ヘッダを立てる必要もありません（立てられないホストでもそのまま動く）。

```
master   ソース一式・レポート
gh-pages  Web ビルドのみ (index.* と .nojekyll)
```

ソースとビルドをブランチで分けているので、master に 43MB のバイナリが混ざりません。**index.wasm** は Godot のランタイムなので、ゲームを更新しても中身は変わりません（変わるのは 190KB 程度の **index.pck** だけ）。

転送量は gzip 後で約 9MB です。

■ 描画も推論も、開いた端末の中で動いている

GitHub Pages は**ファイルを配るだけで**、計算は一切していません。スマホで開けばスマホの GPU が描き、**対戦相手の AI の思考もその端末の中で動いています**。通信は最初のダウンロードだけなので、機内モードでも遊べます。

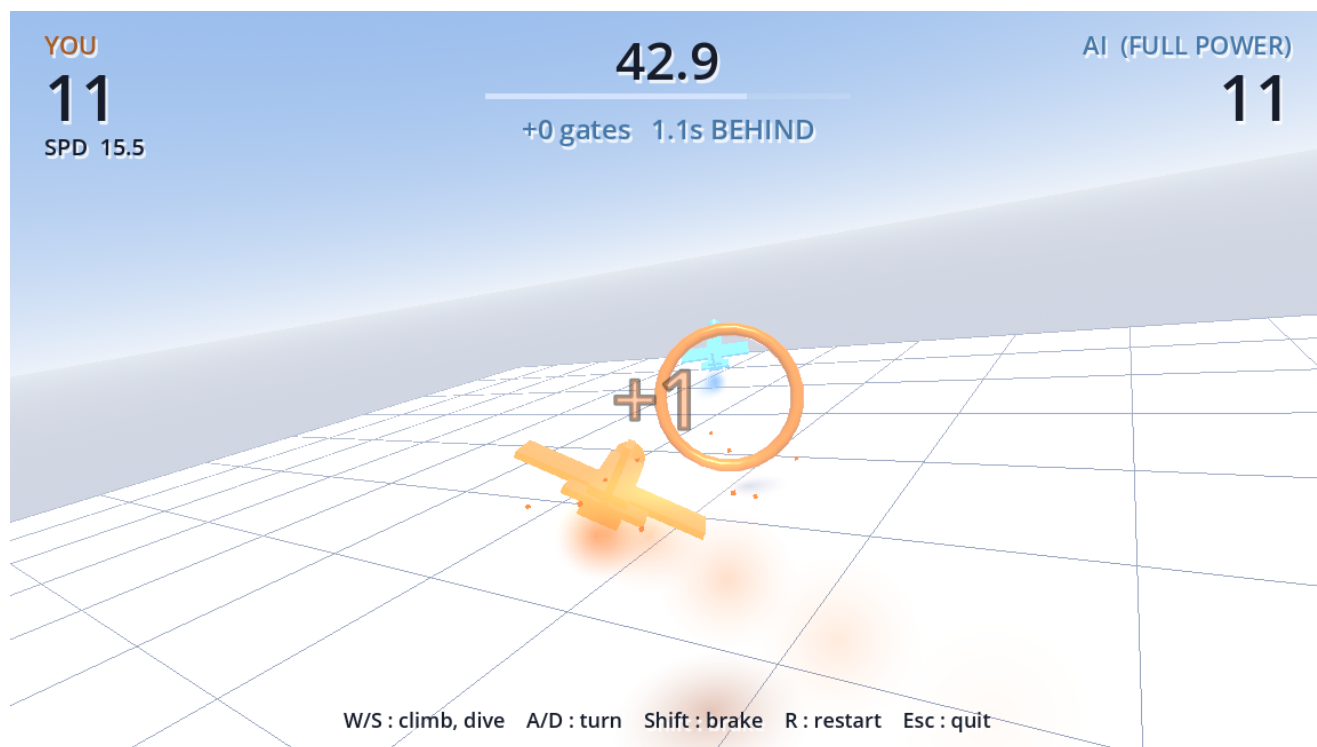
ブラウザは今や、アプリを入れずに 3D ゲームエンジンをそのまま動かせる実行環境になっています。効いているのは **WebAssembly**（C++ で書かれた Godot 本体をほぼネイティブ速度で走らせる）と **WebGL**（GPU に直接描かせる）の 2 つです。

8 つまずいた点

目標が切り替わる瞬間に巨大な偽の報酬が出る。 リング通過・墜落の直後は「目標までの距離」が不連続に跳びます。そのままだと成功したのに罰が入り、AI は「リングをくぐると損をする」と学んでしまいます。切り替えの直後に基準距離を取り直して解消しました。しかも墜落直後は物理サーバー上の位置がまだ更新されていないため、機体の現在位置ではなく**これから戻す予定の座標**から計算する必要がありました。

コース生成のシードを固定したら学習が壊れかけた。ゴースト対戦のために「2本のコースに同じ乱数シードを与える」仕組みを足したところ、既定値のままだと学習用の16コースが全部同じ形になり、しかも毎エピソード同じになってしまいました。「0 = 毎回ランダム」という規約にして回避しています。

落下影が追従カメラでは見えなかった。高度の手がかりとして機体の真下に影を落としたのに、追従カメラは機体の後ろから見ているので真下の地面が画面の下端より外に来てしまいます。一時的に影を派手な色に塗って切り分けました。進行方向の先に投影する対地マーカーに変更しています。



演出のサイズが前作のカメラ前提だった。前作は20m上空から見下ろすカメラですが、今作は機体の10m後ろ。そのまま流用したら「+1」の吹き出しと破片が画面を埋め尽くしました。

エクスポートテンプレートをインストール済みパッケージが消えた。コンテナ環境が途中でリセットされ、1.9GBのテンプレートを再取得する羽目になりました。永続化されないディレクトリに置かれるものは、復旧手順をREADMEに書いておくべきです。

9 学習を速くするための構成



強化学習は「とにかく大量に試す」ことが必要なので、試行を集める速さがそのまま学習時間になります。

手段	内容	効果
コースを並べる	1 プロセスに独立したコースを 16 面置き、16 体を同時に飛ばす	1 回の通信で 16 体ぶんの経験
プロセスを増やす	その Godot を 3 個起動して合計 48 体	CPU コアを使い切る
物理を早送り	描画しないので実時間に縛られる理由がない。40 倍速	60 秒のプレイが 1.5 秒

Godot 側で約 4,200 steps/s、PPO の勾配計算を含めた実効で約 2,950 steps/s。60 万ステップが CPU だけで **3.4 分**でした。

なお **GPU があっても速くなりません**。ネットワークが 5,500 個の重みしかなく小さすぎて、GPU に送って戻す手間の方が計算より大きくなるからです。

まとめ

- **手書きの方策を先に書く**。物差しが無いと学習結果の良し悪しを判断できず、観測や行動の符号ミスにも気づけません
- **小さな MLP で足りるタスクに設計すると、そのままゲームに埋め込める**。ONNX ランタイムが使えなくても、行列積 3 回なら自分で書けます
- **移植は数値で照合する**。「動いているように見える」は当てになりません
- **距離シェーピングと残り時間の観測**が、この規模のタスクでは効きます

ソースとレポート（日本語 PDF・15 ページ）は全部置いてあります。

- 遊ぶ: <https://ms022019.github.io/fly-by-3d/touch/>
- ソース: <https://github.com/ms022019/fly-by-3d>